NumPy Reference Guide

Mathematical Operations, Indexing, and Broadcasting

Data Science Reference

June 16, 2026

Contents

1 Introduction to NumPy

1.1 Definition

NumPy (Numerical Python) is the fundamental library for scientific computing in Python. Its core feature is the `ndarray` (n-dimensional array) object—a fast, fixed-type container for large grids of numbers that offers efficient, vectorized mathematical operations.

1.2 Key Characteristics

- **Vectorized Operations:** Perform calculations on entire arrays without explicit loops
- **Memory Efficient:** Stores homogeneous data types in contiguous memory blocks
- **Broadcasting:** Enables arithmetic operations between arrays of different shapes
- **Integration:** Serves as the foundation for Pandas, Matplotlib, and Scikit-learn

2 NumPy Array vs Python List

2.1 Speed Comparison

NumPy arrays use vectorized operations written in C, processing entire arrays at once. Python lists require element-by-element iteration, which is significantly slower.

Listing 1: Speed Comparison

```
import numpy as np
import time

size = 1_000_000
python_list1 = list(range(size))
python_list2 = list(range(size))
numpy_array1 = np.arange(size)
numpy_array2 = np.arange(size)

# Python List (slow)
start = time.time()
result_list = [python_list1[i] + python_list2[i] for i in range(size)]
python_time = time.time() - start

# NumPy Array (fast)
start = time.time()
result_array = numpy_array1 + numpy_array2
numpy_time = time.time() - start

print(f"NumPy is {python_time/numpy_time:.0f}x faster!")
```

2.2 Memory Comparison

NumPy arrays store homogeneous data types in contiguous memory, while Python lists store pointers to scattered Python objects.

Listing 2: Memory Comparison

```
import numpy as np
import sys

python_int_list = list(range(1000))
numpy_int_array = np.arange(1000)

python_memory = sys.getsizeof(python_int_list)
for element in python_int_list:
    python_memory += sys.getsizeof(element)
```

```python

numpy_memory = numpy_int_array.nbytes

print(f"Python List: {python_memory:,} bytes")
print(f"NumPy Array: {numpy_memory:,} bytes")
print(f"NumPy uses {python_memory/numpy_memory:.1f}x less memory")
```

2.3 Comparison Table

Aspect	Python List	NumPy Array
Data Types	Heterogeneous	Homogeneous
Speed	Slow	Fast
Memory	High	Low
Resizing	Easy (.append)	Creates new array
Math Operations	Explicit loops	Vectorized
Best For	General programming	Scientific computing

Table 1: Python List vs NumPy Array

3 Fancy Indexing and Boolean Indexing

3.1 Fancy Indexing

3.1.1 Definition

Fancy Indexing uses integer arrays or lists as indices to select arbitrary elements from a NumPy array in any order.

3.1.2 Key Characteristics

- Uses integer indices (positions)
- Can select elements in any order
- Can repeat the same index multiple times
- Returns a copy of the data

Listing 3: Fancy Indexing

```python
import numpy as np

# 1D Array
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
indices = [0, 2, 4, 6]
result = arr[indices]  # [10, 30, 50, 70]

# Select in any order
indices_reverse = [6, 4, 2, 0]
result_reverse = arr[indices_reverse]  # [70, 50, 30, 10]

# Repeat indices
indices_repeat = [0, 0, 3, 3, 5]
result_repeat = arr[indices_repeat]  # [10, 10, 40, 40, 60]

# 2D Array
arr_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
rows = [0, 2]
cols = [1, 3]
result = arr_2d[rows, cols]  # [2, 12]
```

3.2 Boolean Indexing

3.2.1 Definition

Boolean Indexing uses a boolean array (True/False values) to select elements based on conditions.

3.2.2 Key Characteristics

- Uses boolean masks (True/False)
- Created through comparison operations
- Can combine multiple conditions with &, —,
- Returns a copy of the data

Listing 4: Boolean Indexing

```
import numpy as np

# 1D Array
arr = np.array([10, 15, 20, 25, 30, 35, 40])
mask = arr > 25
result = arr[mask]  # [30, 35, 40]

# Multiple conditions
mask_multiple = (arr >= 20) & (arr <= 35)
result = arr[mask_multiple]  # [20, 25, 30, 35]

# OR condition
mask_or = (arr < 15) | (arr > 30)
result = arr[mask_or]  # [10, 35, 40]

# 2D Array - Data Cleaning
grades = np.array([[85, 92, 78], [65, 70, 95], [45, 60, 55]])
grades[grades < 60] = 60  # Replace failing grades
high_scores = grades[grades > 80]  # [85, 92, 95]
```

3.3 Comparison Table

Aspect	Fancy Indexing	Boolean Indexing
Index Type	Integer arrays	Boolean arrays
Use Case	Specific positions	Filtering by conditions
Order Control	Full control	Original order
Conditional Selection	No	Yes
Result Shape	Shape of index array	1D array

Table 2: Fancy Indexing vs Boolean Indexing

4 Broadcasting in NumPy

4.1 Definition

Broadcasting is NumPy's mechanism that allows arithmetic operations between arrays of different shapes. The smaller array is "broadcast" across the larger array.

4.2 The 3 Rules of Broadcasting

4.2.1 Rule 1: Matching Dimensions from the Right

When operating on two arrays, NumPy compares their shapes element-wise starting from the trailing (rightmost) dimension and moving left.

4.2.2 Rule 2: Dimension Compatibility

Two dimensions are compatible when:

- They are equal, OR
- One of them is 1

4.2.3 Rule 3: Error if Incompatible

If Rule 2 fails, broadcasting fails with a ValueError.

4.3 Broadcasting Examples

Listing 5: Broadcasting Examples

```
import numpy as np

# Scalar Broadcasting
arr = np.array([1, 2, 3, 4, 5])
result = arr + 10  # [11, 12, 13, 14, 15]

# Row Broadcasting
matrix = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
row = np.array([10, 20, 30, 40])
result = matrix + row
# [[11, 22, 33, 44],
#  [15, 26, 37, 48],
#  [19, 30, 41, 52]]

# Column Broadcasting
column = np.array([[10], [20], [30]])
result = matrix + column
# [[11, 12, 13, 14],
#  [25, 26, 27, 28],
#  [39, 40, 41, 42]]

# Column + Row (creates grid)
col = np.array([[1], [2], [3], [4]])  # (4, 1)
row = np.array([10, 20, 30])          # (3,)
result = col + row
# [[11, 21, 31],
#  [12, 22, 32],
#  [13, 23, 33],
#  [14, 24, 34]]

# Broadcasting Visualization
print("Shape Compatibility:")
print("(3,4) + (4,)  -> Compatible    ")
print("(3,4) + (3,1) -> Compatible    ")
print("(3,4) + (1,4) -> Compatible    ")
print("(3,4) + (3,5) -> Incompatible    ")
```

4.4 Compatibility Matrix

	1	2	3	4	5
1					
2					
3					
4					
5					

Table 3: Dimension Compatibility Matrix

Table 4: *
= Compatible (can broadcast), = Incompatible

5 Mathematical Operations in NumPy

5.1 Basic Arithmetic Operations

Operator	NumPy Function	Description
+	np.add()	Element-wise addition
-	np.subtract()	Element-wise subtraction
*	np.multiply()	Element-wise multiplication
/	np.divide()	Element-wise division
**	np.power()	Element-wise exponentiation
%	np.mod()	Element-wise modulus

Listing 6: Basic Arithmetic

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([2, 4, 6])

print(a + b)            # [12, 24, 36]
print(np.add(a, b))     # [12, 24, 36]
print(a - b)            # [8, 16, 24]
print(a * b)            # [20, 80, 180]
print(a / b)            # [5, 5, 5]
print(a ** 2)           # [100, 400, 900]
print(a % b)            # [0, 0, 0]
```

5.2 Trigonometric Functions

Listing 7: Trigonometric Functions

```
arr = np.array([0, np.pi/2, np.pi])

# Basic trig functions
print(np.sin(arr))    # [0, 1, 1.2246468e-16]
print(np.cos(arr))    # [1, 6.1232339e-17, -1]
print(np.tan(arr))    # [0, 1.6331239e+16, -1.2246468e-16]

# Inverse trig functions
print(np.arcsin([0, 1]))    # [0, 1.57079633]
print(np.arccos([1, 0]))    # [0, 1.57079633]
print(np.arctan([0, 1]))    # [0, 0.78539816]
```

5.3 Exponential and Logarithmic Functions

Listing 8: Exponential and Logarithmic

```
arr = np.array([1, 2, 3])

# Exponential
print(np.exp(arr))      # [2.71828183, 7.3890561, 20.08553692]
print(np.expm1(arr))    # [1.71828183, 6.3890561, 19.08553692]

# Logarithms
print(np.log(arr))      # [0, 0.69314718, 1.09861229]
print(np.log10(arr))    # [0, 0.30102999, 0.47712125]
print(np.log2(arr))     # [0, 1, 1.5849625]
```

5.4 Rounding Functions

Listing 9: Rounding Functions

```
arr = np.array([3.14159, 2.71828, 1.41421, 2.5])

print(np.around(arr, decimals=2))  # [3.14, 2.72, 1.41, 2.5]
print(np.rint(arr))                 # [3., 3., 1., 2.]
print(np.floor(arr))                # [3., 2., 1., 2.]
print(np.ceil(arr))                 # [4., 3., 2., 3.]
print(np.trunc(arr))                # [3., 2., 1., 2.]
```

5.5 Statistical and Aggregate Functions

Function	Description
np.mean(), np.median()	Arithmetic mean and median
np.sum(), np.cumsum()	Sum and cumulative sum
np.std(), np.var()	Standard deviation and variance
np.min(), np.max()	Minimum and maximum values
np.percentile(), np.quantile()	Percentiles or quantiles
np.prod(), np.cumprod()	Product and cumulative product
np.diff()	Discrete differences

Listing 10: Statistical Functions

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])

print(np.mean(matrix))          # 3.5
print(np.mean(matrix, axis=0))  # [2.5, 3.5, 4.5]
print(np.sum(matrix, axis=0))   # [5, 7, 9]
print(np.cumsum(matrix))        # [1, 3, 6, 10, 15, 21]
print(np.std(matrix))           # 1.7078
print(np.var(matrix))           # 2.9167
print(np.min(matrix))           # 1
print(np.max(matrix))           # 6
print(np.median(matrix))        # 3.5
print(np.percentile(matrix, 25)) # 2.25
print(np.prod(matrix))          # 720
print(np.cumprod(matrix))       # [1, 2, 6, 24, 120, 720]
```

5.6 Comparison and Extrema Functions

Listing 11: Comparison Functions

```
a = np.array([1, 5, 3, 9, 2])
b = np.array([2, 4, 6, 8, 0])

print(np.maximum(a, b))       # [2, 5, 6, 9, 2]
print(np.minimum(a, b))       # [1, 4, 3, 8, 0]

arr = np.array([1, 5, 10, 15, 20])
print(np.clip(arr, 3, 12))    # [3, 5, 10, 12, 12]
print(np.absolute([-1, 2, -3]))  # [1, 2, 3]
print(np.sign([-1, 0, 1]))     # [-1, 0, 1]
print(np.square([1, 2, 3]))    # [1, 4, 9]
print(np.sqrt([1, 4, 9]))      # [1, 2, 3]
```

5.7 Complex Number Operations

Listing 12: Complex Numbers

```
arr = np.array([1+2j, 3+4j, 5+6j])

print(np.real(arr))     # [1., 3., 5.]
print(np.imag(arr))     # [2., 4., 6.]
print(np.conj(arr))     # [1.-2.j, 3.-4.j, 5.-6.j]
print(np.absolute(arr))  # [2.236, 5., 7.810]
```

6 Sigmoid Function

6.1 Definition

The sigmoid function maps any real-valued number to a value between 0 and 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

6.2 Implementation

Listing 13: Sigmoid Function

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

x = np.array([-2, -1, 0, 1, 2])
y = sigmoid(x)
print(x)  # [-2, -1, 0, 1, 2]
print(y)  # [0.1192, 0.2689, 0.5, 0.7311, 0.8808]

# Property: sigmoid(-x) = 1 - sigmoid(x)
print(sigmoid(2) + sigmoid(-2))  # 1.0
print(sigmoid(10))   # 0.99995
print(sigmoid(-10))  # 0.000045
```

7 Mean Squared Error (MSE)

7.1 Definition

MSE measures the average squared difference between predicted and actual values:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

7.2 Implementation

Listing 14: Mean Squared Error

```python
import numpy as np

def mean_squared_error(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

y_true = np.array([1, 2, 3, 4, 5])
y_pred = np.array([1.2, 1.8, 3.1, 3.9, 5.2])

mse = mean_squared_error(y_true, y_pred)
print(f"MSE: {mse:.4f}")  # MSE: 0.0300

# Manual calculation
squared_errors = (y_true - y_pred) ** 2
manual_mse = np.mean(squared_errors)
print(f"Manual MSE: {manual_mse:.4f}")  # 0.0300
```

8 Binary Cross Entropy

8.1 Definition

Binary Cross Entropy measures the performance of a binary classification model:

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

8.2 Implementation

Listing 15: Binary Cross Entropy

```python
import numpy as np

def binary_cross_entropy(y_true, y_pred):
    epsilon = 1e-15  # Avoid log(0)
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

y_true = np.array([1, 0, 1, 0, 1])
y_pred = np.array([0.9, 0.1, 0.8, 0.2, 0.95])

bce = binary_cross_entropy(y_true, y_pred)
print(f"BCE Loss: {bce:.4f}")

# Perfect predictions
perfect_pred = y_true.astype(float)
perfect_bce = binary_cross_entropy(y_true, perfect_pred)
print(f"BCE with perfect predictions: {perfect_bce:.4f}")
```

9 Working with Missing Values (NaN)

9.1 Definition

NaN (Not a Number) represents missing or undefined numerical values in NumPy arrays.

9.2 Common Operations

Listing 16: Working with NaN

```python
import numpy as np

arr = np.array([1, 2, np.nan, 4, 5, np.nan, 7])

# Checking for NaN
print(np.isnan(arr))  # [False False True False False True False]
print(np.isnan(arr).sum())  # 2

# Removing NaN values
print(arr[~np.isnan(arr)])  # [1, 2, 4, 5, 7]

# Replace NaN with a value
arr_filled = np.nan_to_num(arr, nan=0)
print(arr_filled)  # [1, 2, 0, 4, 5, 0, 7]

# Statistical operations
print(np.nanmean(arr))   # 3.8
print(np.nanstd(arr))    # 2.1354
print(np.nansum(arr))    # 19.0

# Fill NaN with mean
mean_val = np.nanmean(arr)
arr_filled_mean = np.where(np.isnan(arr), mean_val, arr)
print(arr_filled_mean)  # [1, 2, 3.8, 4, 5, 3.8, 7]
```

9.3 None vs NaN

Feature	**None**	**np.nan**
Type	NoneType	float
Mathematical Operations	Error	Results in NaN
Comparison	Works (is None)	Always False
Common Usage	Missing data in Python	Missing data in NumPy
Memory	Python object	Float

Table 5: None vs NaN

Listing 17: None vs NaN

```python
import numpy as np

arr_none = np.array([1, 2, None, 4])  # Object array
print(arr_none)  # [1, 2, None, 4]

arr_nan = np.array([1, 2, np.nan, 4])  # Float array
print(arr_nan)  # [1., 2., nan, 4.]

# Comparison with NaN
print(arr_nan == np.nan)  # [False, False, False, False]

# Checking for missing values
def is_missing(x):
    return np.isnan(x) if isinstance(x, float) else x is None
```

10 Visualization with Matplotlib

10.1 Definition

Matplotlib is the foundational plotting library in Python, providing publication-quality static, animated, and interactive visualizations.

10.2 Basic Plotting

Listing 18: Matplotlib Line Plot

```
import matplotlib.pyplot as plt
import numpy as np

# Create data
x = np.linspace(0, 10, 20)
y = x ** 2

# Create plot
plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', linestyle='--', color='red', label='y = x  ')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('Simple Line Plot')
plt.legend()
plt.grid(True)
plt.show()

# Multiple plots
x = np.linspace(0, 2*np.pi, 100)
plt.plot(x, np.sin(x), label='sin(x)')
plt.plot(x, np.cos(x), label='cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Sine and Cosine Functions')
plt.legend()
plt.grid(True)
plt.show()
```

10.3 Subplots and Multiple Figures

Listing 19: Multiple Plots

```
import matplotlib.pyplot as plt
import numpy as np

# Create subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
x = np.linspace(-5, 5, 100)

# Plot 1: Linear
axes[0, 0].plot(x, x, 'b-', label='y=x')
axes[0, 0].set_title('Linear')
axes[0, 0].grid(True)

# Plot 2: Quadratic
axes[0, 1].plot(x, x**2, 'r-', label='y= x  ')
axes[0, 1].set_title('Quadratic')
axes[0, 1].grid(True)

# Plot 3: Sine
axes[1, 0].plot(x, np.sin(x), 'g-', label='y=sin(x)')
axes[1, 0].set_title('Sine')
axes[1, 0].grid(True)

# Plot 4: Exponential
axes[1, 1].plot(x, np.exp(x), 'm-', label='y=e^x')
axes[1, 1].set_title('Exponential')
axes[1, 1].grid(True)

plt.tight_layout()
plt.show()
```

10.4 Histograms and Bar Charts

Listing 20: Histograms and Bar Charts

```
import matplotlib.pyplot as plt
import numpy as np

# Histogram
data = np.random.normal(0, 1, 1000)
plt.figure(figsize=(8, 4))
plt.hist(data, bins=30, alpha=0.7, color='blue', edgecolor='black')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram of Normal Distribution')
plt.grid(True, alpha=0.3)
plt.show()

# Bar chart
categories = ['A', 'B', 'C', 'D', 'E']
values = [25, 40, 30, 55, 45]
plt.figure(figsize=(8, 4))
plt.bar(categories, values, color=['red', 'blue', 'green', 'orange', 'purple'])
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Chart Example')
plt.show()
```

10.5 Scatter Plots

Listing 21: Scatter Plots

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data
np.random.seed(42)
x = np.random.randn(100)
y = 2 * x + 1 + 0.5 * np.random.randn(100)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, alpha=0.6, color='blue', label='Data points')
plt.plot(x, 2*x + 1, 'r-', label='True line')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Scatter Plot with Regression Line')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

10.6 Plot Customization

Listing 22: Plot Customization

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(10, 6))

# Customize plot
plt.plot(x, y, 'b-', linewidth=2, label='sin(x)')
plt.fill_between(x, y, alpha=0.2, color='blue')
```

```
12
13 # Customize axes
14 plt.xlabel('X Axis', fontsize=12, fontweight='bold')
15 plt.ylabel('Y Axis', fontsize=12, fontweight='bold')
16 plt.title('Customized Plot', fontsize=14, fontweight='bold')
17
18 # Customize ticks
19 plt.xticks(np.arange(0, 11, 2))
20 plt.yticks(np.arange(-1, 1.5, 0.5))
21
22 # Add grid with customization
23 plt.grid(True, linestyle='--', alpha=0.7)
24
25 # Add legend
26 plt.legend(loc='upper right', fontsize=10)
27
28 # Add annotations
29 plt.annotate('Maximum', xy=(np.pi/2, 1), xytext=(np.pi/2+0.5, 1.2),
30              arrowprops=dict(facecolor='red', shrink=0.05))
31
32 plt.show()
```

11 NumPy Cheat Sheet

11.1 Array Creation

Function	Description
np.array([1,2,3])	Create array from list
np.zeros((3,4))	Array of zeros
np.ones((2,3))	Array of ones
np.eye(5)	Identity matrix
np.arange(0,10,2)	Range with step
np.linspace(0,1,10)	Linearly spaced values
np.random.randn(3,4)	Random normal distribution
np.random.randint(0,10,size=(3,4))	Random integers

Table 6: Array Creation

11.2 Array Attributes and Operations

Attribute/Operation	Description
arr.shape	Get array dimensions
arr.size	Total number of elements
arr.dtype	Data type of elements
arr.ndim	Number of dimensions
arr.reshape(2,3)	Reshape array
arr.flatten()	Flatten to 1D
arr.T	Transpose
arr.T @ arr	Matrix multiplication

Table 7: Array Attributes and Operations

11.3 Indexing and Slicing

Operation	Example
Element access	arr[2, 3]
Row access	arr[1, :]
Column access	arr[:, 2]
Slicing	arr[1:4, 2:5]
Fancy indexing	arr[[0,2,4], :]
Boolean indexing	arr[arr ¿ 5]

Table 8: Indexing and Slicing

11.4 Mathematical Operations

Operation	Function
Addition	np.add(), arr + 5
Multiplication	np.multiply(), arr * 2
Power	np.power(), arr ** 2
Square root	np.sqrt(arr)
Exponential	np.exp(arr)
Logarithm	np.log(arr), np.log10(arr)
Trigonometric	np.sin(arr), np.cos(arr), np.tan(arr)

Table 9: Mathematical Operations

11.5 Statistical Operations

Operation	Function
Mean	np.mean(arr), np.nanmean(arr)
Median	np.median(arr)
Sum	np.sum(arr), np.nansum(arr)
Standard deviation	np.std(arr), np.nanstd(arr)
Variance	np.var(arr), np.nanvar(arr)
Minimum	np.min(arr)
Maximum	np.max(arr)
Percentile	np.percentile(arr, 50)

Table 10: Statistical Operations

11.6 Aggregation and Reductions

Operation	Function
Cumulative sum	np.cumsum(arr)
Cumulative product	np.cumprod(arr)
Differences	np.diff(arr)
Sort	np.sort(arr), arr.sort()
Unique values	np.unique(arr)
Concatenate	np.concatenate([arr1, arr2])
Stack	np.vstack(), np.hstack()
Split	np.split(arr, 3)

Table 11: Aggregation and Reductions

11.7 Common Broadcasting Patterns

Pattern	Example
Scalar + Array	arr + 5
Row + Matrix	matrix + row (1×4)
Column + Matrix	matrix + col (3×1)
Row + Column	col(4×1) + row(1×3)

Table 12: Common Broadcasting Patterns

11.8 Handling Missing Data

Operation	Function
Check NaN	np.isnan(arr)
Count NaN	np.isnan(arr).sum()
Remove NaN	arr[np.isnan(arr)]
Replace NaN	np.nan$_to_n um(arr, nan = 0)$
Safe mean	np.nanmean(arr)
Safe sum	np.nansum(arr)

Table 13: Handling Missing Data

11.9 Common Loss Functions

Loss Function	Formula
MSE	np.mean((y_true - y_pred) ** 2)
MAE	np.mean(np.abs(y_true - y_pred))
Binary Cross Entropy	-np.mean(y*np.log(p) + (1-y)*np.log(1-p))

Table 14: Common Loss Functions

11.10 Quick Tips

- Use **broadcasting** instead of loops for faster code
- Always handle **NaN** values before statistical operations
- Use **views** (arr.view()) instead of copies (arr.copy()) when possible
- Choose appropriate data types (`float32` vs `float64`) for memory efficiency
- Use **vectorized operations** for better performance
- Check array shapes with `arr.shape` before operations
- Use **broadcasting rules** to understand array compatibility